

Cross-engine extraction comparison report

Introduction

The British Columbia government's **SDPR Monthly Report** is a one-page form that income-assistance recipients fill out monthly. Each form has 74 fields covering applicant and spouse names, signatures, dates, SINS, phone numbers, a free-form comment box, two parallel columns of 18 income amounts (one column per person), and 14 yes/no checkbox pairs. Most fields are filled by hand. The forms come in via scanner or phone photo and need to be turned into structured data that downstream eligibility processing can consume.

This report is a head-to-head comparison of **seven different ways to do that extraction**, plus a synthesised ensemble. Each engine reads the same set of 40 sample forms, returns the same 74 fields per form, and is scored the same way against the same ground truth. The forms span hand-filled real-world samples (varying handwriting styles, including pencil and a phone photo), synthetically generated samples, and two deliberately-hard edge cases (a blank form and one with coffee stains over the data).

What the engines have in common. Every engine takes an image (or PDF page) of the form and returns a JSON object with 74 field values. None of them are trained on this specific form — they all work from a description of the schema we provide. Re-scoring is strict: an answer is either an exact match to ground truth (with small allowances for known-equivalent format variants on dates, SINS, and phone numbers) or it isn't.

What the engines differ on. They differ on the OCR layer (or whether there is one), the generative model used to interpret the text and fill the schema, whether the image itself is shown to the generative model, and how the prompt and schema are presented to it. Those four levers turn out to drive most of the accuracy differences we see — picking the right combination matters more than picking the latest model.

Headline finding. A custom pipeline that pairs **Azure Document Intelligence's** `prebuilt-layout` **OCR with a Foundry-deployed GPT-5.2 chat-completions call** — sending both the image and the OCR markdown to the model — is the strongest single engine on this dataset, beating the managed Azure Content Understanding (CU) service on most aggregates *even though CU runs the same gpt-5.2 model internally*. The gap is structural, not model-driven: CU's OCR is tuned for grounding-strict behaviour that drops marginal handwriting, and CU never shows the image to the LLM. Our pipeline sees both the image *and* the OCR text, which lets the model recover content CU's stricter OCR drops — at the cost of higher per-call inference (image tokens are expensive). Full deep-dive is in the [E08 section](#).

How to read the rest of this document. The headline aggregate metrics at the top give the bottom line. The per-category and per-field views show where each engine wins and loses. The per-engine deep-dive sections explain *why* each engine behaves the way it does. The Reflection at the end ties the patterns together. Appendix A documents the ensemble (E06).

What this report compares

This report compares seven end-to-end OCR / form-extraction approaches against the same set of monthly-report documents, plus a synthesised ensemble that combines the original five (E06). Each engine extracts the same 74 fields per document; every engine is scored the same way against the same ground truth so the numbers are directly comparable.

The seven engines include three different vision-language models running through the same VLM + OCR hybrid pipeline (E05 / E07 / E08), which lets us isolate the contribution of the underlying model from the contribution of the pipeline architecture.

The dataset

The dataset contains **40 SDPR Monthly Report documents** — the form that anchors this comparison:

- **21 hand-filled real-world samples** from the HR0081 series and a "Fake N" series — pen-on-paper handwriting from a range of people, including one sample filled in pencil, one phone-camera photo with the form's background visible, and two intentionally hard samples (one blank, one with coffee-stain obscuration). These are the closest analogue we have to documents that would arrive in production. (A handful of fields on some real samples — printed labels, pre-typed identifiers — are typed rather than hand-written, but the data entries themselves are hand-written.)
- **10 hand-filled "manual sample" forms** — additional handwriting samples, captured cleanly.
- **9 synthetic samples** ("synth-full", "synth-no-spouse", "synth-regular" series) — synthetically-generated forms with hand-written field values. The handwriting style is more uniform than the real-world samples, but the data itself is handwriting, not typed text.

Caveat on generalisability: the numbers in this report are computed against 40 sample documents. **These results may not match what you would see on production data; treat the rankings as directionally correct rather than as absolute production accuracy estimates.** Production submissions will have different handwriting styles, scan qualities, lighting conditions, and edge cases that this dataset does not cover. The engine-level patterns (which engines win which categories, where the failure modes are) are likely to hold, but the absolute F1 / precision / recall numbers will move on a larger and more representative corpus.

Engines compared

- **E00 — Azure Document Intelligence custom template.** A supervised, form-specific model: a labelled template is trained against the SDPR form and inferred against new images. This is the same workflow that produced the **V1 Report**.
- **E02 — Mistral on Azure AI Foundry.** A general-purpose document AI model. The engine first runs OCR over the page, then a Mistral generative pass extracts structured fields against a JSON schema.
- **E03 — Azure Content Understanding (CU) + GPT-5.2.** A managed two-stage Microsoft service. Stage 1 is a specialized OCR/Layout model (the same Document Intelligence `prebuilt-layout` family, tuned for stricter grounding) that produces a layout-aware Markdown representation of the page. Stage 2 sends that Markdown plus the analyzer schema (field names, types, descriptions) to a customer-deployed GPT-5.2 chat model and gets a structured JSON response back. **The image itself is never sent to the LLM** — stage 2 is text-only.
- **E04 — GPT-5.4 vision-language model (direct).** The page image is sent directly to GPT-5.4 with a strict JSON-schema response format. No separate OCR step.
- **E05 — GPT-5.4 VLM + Azure DI layout (hybrid).** Two-pass: Azure DI's layout reader transcribes the page to markdown, then GPT-5.4 sees both the raw image and the OCR text, with a prompt that tells it to trust the image when the OCR text disagrees.
- **E07 — GPT-4o VLM + Azure DI layout (hybrid).** Same pipeline as E05; the only difference is the generative model — GPT-4o instead of GPT-5.4. Lets us isolate the model contribution on the hybrid pipeline.
- **E08 — GPT-5.2 VLM + Azure DI layout (hybrid).** Same pipeline as E05 and E07; the generative model is GPT-5.2 — the same model E03's Content Understanding uses internally. The third leg of the same-pipeline model bake-off.
- **E06 — ensemble combiner.** Not itself a deployed engine. It takes the predictions produced by the original five (E00–E05) and picks the per-field value using a weighted-majority vote, where each engine's vote weight is its own historical per-field accuracy. Full details in [Appendix A](#). E07 and E08 are not included in the ensemble.

E01 (Azure DI **Neural** custom model — a deep-learning model that infers field positions instead of relying on labelled bounding boxes the way E00's Template model does) is being worked on separately and is not included in this comparison.

How the metrics are computed

Each document has 74 fields with known correct values. Every engine returns the same 74 fields with whatever values it extracted. For each field on each document, we ask: did the engine's value match the correct value? The comparison is strict — values either match exactly or they don't (with a small allowance for known-equivalent format variants on dates / SINs / phones, so e.g. 2025–Nov–12 and 2025-11-12 are both accepted for a date written on the form as 2025–Nov–12).

The atomic units: TP, FP, FN

Every per-field comparison falls into one of four buckets:

case	correct value (GT)	engine prediction	classification	counts
match	value	matches GT	TP (true positive) — engine read the field correctly	TP += 1
deletion	value	null / missing	FN only (false negative) — the correct value did not appear in the output	FN += 1
insertion	null	non-null	FP only (false positive) — the engine produced a value that wasn't there	FP += 1
substitution	value	non-null, wrong	FP + FN — the engine BOTH produced a wrong value (precision miss) AND missed the correct value (recall miss)	FP += 1, FN += 1

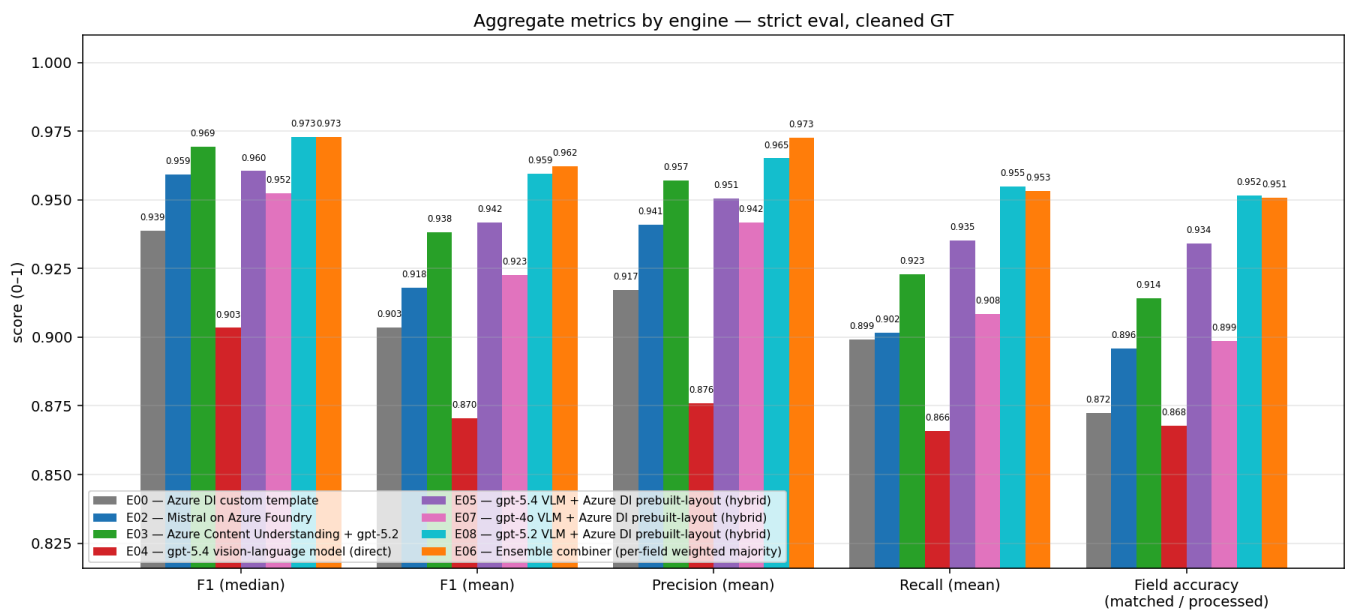
These definitions match the standard OCR / information-extraction formulation. A substitution counts on both sides because it's two errors in one place: a wrong value was produced *and* the correct value was missed.

The derived metrics

- **Precision (per sample)** = $TP / (TP + FP)$. "*Of the answers the engine gave, what fraction was correct?*" Low precision means the engine produces wrong values or hallucinates values for blank fields.
- **Recall (per sample)** = $TP / (TP + FN)$. "*Of the values that were actually on the form, what fraction did the engine read?*" Low recall means the engine misses fields or returns nothing when it should have produced a value.
- **F1 (per sample)** = $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$. Harmonic mean of precision and recall, designed to drop sharply if either input is weak. F1 is the single-number summary that punishes lopsided systems — an engine with high precision but low recall (or vice versa) gets a worse F1 than an engine that balances both.
- **matchedFields (per sample)** = TP. Absolute count of correctly-extracted fields out of 74 (or 52 for documents with no spouse).
- **Field accuracy (per engine)** = total matched fields summed across all 40 documents, divided by the total number of GT fields evaluated (2,852). The intuitive "*if you pick a random field on a random document, how often is it right?*" metric.
- **FP.mean (per engine)** = average number of false-positive predictions per sample. With 74 fields per sample, typical FP.mean values are in the 0–10 range — counts wrong-value substitutions plus values produced for genuinely blank cells plus any fields the engine emitted outside the schema.

Per-engine aggregates (median, mean) are computed across the 40 per-sample F1 / precision / recall values.

Headline aggregate metrics



How to read: every metric on this chart runs from 0 to 1 (1 is perfect). The Y-axis is zoomed (the visible range starts well above 0) so that small differences between engines are visible — the absolute differences look modest here, but they compound: a 2 pp difference in field accuracy is roughly 1.5 extra correct fields per document, or several hundred extra correct extractions across a batch of 10,000 documents. Each group of bars is one metric; each colour is one engine (full names in the legend). Higher is better.

	E00 (DI template)	E02 (Mistral)	E03 (CU + gpt-5.2)	E04 (gpt-5.4 direct)	E05 (gpt-5.4 hybrid)	E07 (gpt-4o hybrid)	E08 (gpt-5.2 hybrid)	E06 (ensemble)
F1 (median)	0.939	0.959	0.969	0.903	0.961	0.952	0.973	0.973
F1 (mean)	0.903	0.918	0.938	0.870	0.942	0.923	0.960	0.962
Precision (mean)	0.917	0.941	0.957	0.876	0.951	0.942	0.965	0.973
Recall (mean)	0.899	0.902	0.923	0.866	0.935	0.909	0.955	0.953
matchedFields (median)	66	69	70	66	71	68	71	71
Field accuracy (matched / processed)	0.872 (2488/2852)	0.896 (2555/2852)	0.914 (2607/2852)	0.868 (2475/2852)	0.934 (2664/2852)	0.899 (2563/2852)	0.952 (2714/2852)	0.951 (2712/2852)
False positives (mean per sample)	5.60	4.05	2.88	8.48	3.38	4.00	2.50	1.93

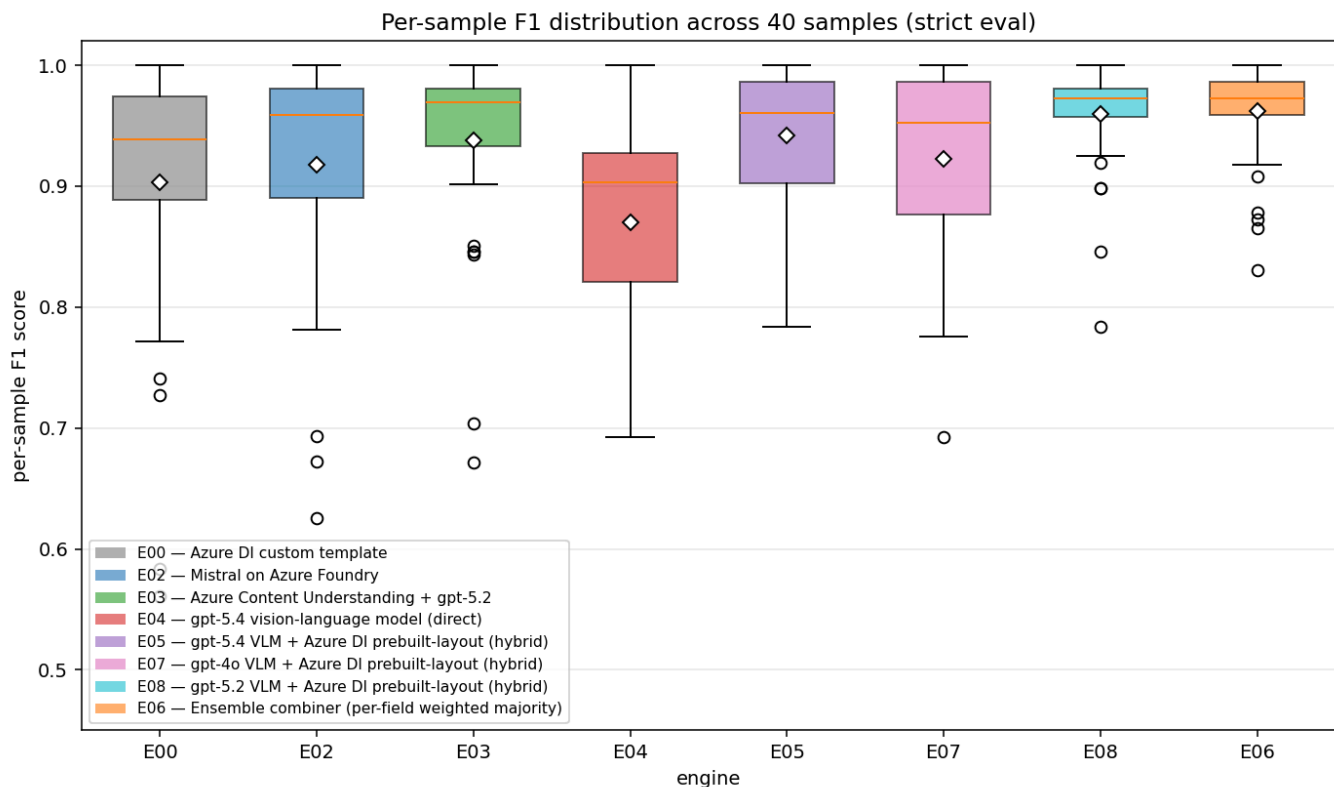
Four observations to keep in mind throughout the rest of the document:

- E08 (gpt-5.2 on the hybrid pipeline) is the strongest single engine on every accuracy aggregate.** F1.mean 0.960, F1.median 0.973, precision.mean 0.965, recall.mean 0.955, FP.mean 2.50. It beats E03 — which uses the same gpt-5.2 generative model internally — by 2.2 pp on F1.mean and 3.2 pp on recall. The gap is structural: CU's stage-1 OCR is tuned for stricter grounding than the standalone DI prebuilt-layout E08 uses, and CU's stage 2 is text-only (image never reaches the LLM), while E08 sends image + OCR Markdown together. **E08's accuracy lead is bought at higher**

per-call inference cost (image tokens add ~2–3× to the LLM input bill). See the [E08 deep dive](#) for the full mechanism and cost breakdown.

2. **The same-pipeline model bake-off (E05 / E07 / E08) gives a clear model ranking on this task: gpt-5.2 > gpt-5.4 > gpt-4o.** With pipeline, prompt, and field-descriptions held constant, gpt-5.2 wins by 1.8 pp F1.mean over gpt-5.4 and 3.7 pp over gpt-4o. Counter-intuitive ordering (the older model number wins), but consistent: gpt-5.2 was tuned for Content Understanding's structured-extraction workload, which is essentially the same shape as this benchmark.
3. **E04 (gpt-5.4 VLM-direct) is meaningfully weaker than its peers.** Lowest precision (0.876) and highest FP.mean (8.48) — the engine substitutes wrong values for fields it isn't sure about. F1.mean 0.870 trails the hybrid-pipeline siblings by 5–9 pp. The OCR layer in front of the same VLM closes that gap entirely (compare E04 vs E05).
4. **The original ensemble (E06) still measurably beats every single engine on FP.mean and precision** (1.93 / 0.973). On F1.mean (0.962) and F1.median (0.973) it is roughly tied with E08 alone. The ensemble's main remaining advantage is its lower wrong-value substitution rate — meaningful, but the gap to a single engine has narrowed. Note: the existing E06 ensemble was built on E00–E05 only; adding E07/E08 to the ensemble pool would likely shift the headroom story but isn't done in this report.

Per-sample F1 distribution



How to read: Each box summarises the distribution of one engine's per-sample F1 scores across the 40 documents.

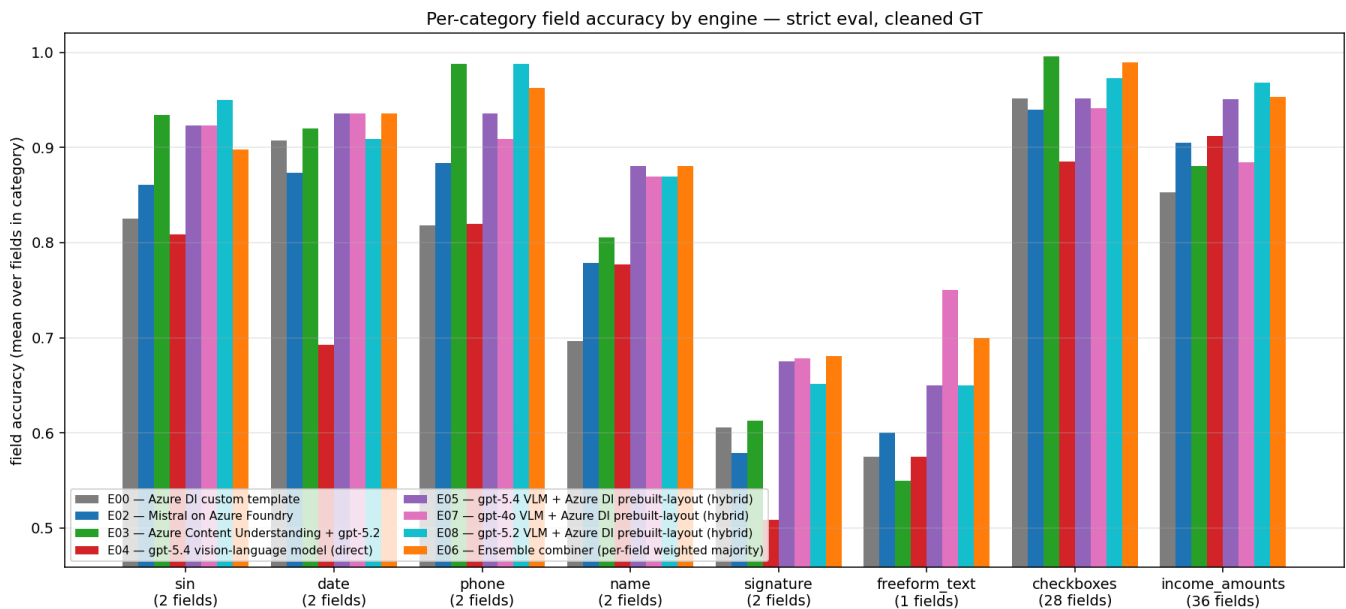
- The **box** spans the 25th to 75th percentile — the middle half of samples.
- The **line inside the box** is the median sample.
- The **white diamond** is the mean sample.
- The **whiskers** (the thin lines extending above and below the box) reach to the most-extreme samples that are still "within reach" of the bulk of the data — specifically, up to 1.5× the box's height (the interquartile range) above and below the box edges. This is the standard Tukey convention for box plots; it's not based on standard

deviation. Any sample further out than that shows up as a separate dot — an *outlier*, meaning a sample that is unusually bad (or unusually good) compared to the engine's typical performance.

- Higher boxes are better. Tighter (shorter) boxes mean the engine performs consistently across samples.

The bottom of each box and the whiskers tell the worst-case story. E04 has the lowest whisker and widest box — its performance varies a lot across samples. E00 also has a wide box (the custom template either nails a sample or struggles meaningfully). E07 (gpt-4o on the hybrid pipeline) widens out vs E05/E08 — its 81 blank outlier sits noticeably lower than any other engine's worst case. **E08 has the tightest distribution of any single engine** (stdDev 0.042 vs E05's 0.060 vs E07's 0.078); its box sits highest on the chart and its whiskers don't reach as far down as E03's or E05's.

Per-category field accuracy



How to read: Each group of bars is one field category; each colour is one engine. The Y-axis is zoomed so the differences are visible. The 74 schema fields are grouped into 8 categories: sin (2 fields), date (2), phone (2), name (2), signature (2), freeform_text (explain_changes, 1 field), checkboxes (28), and income_amounts (37 numeric fields).

category	n fields	E00 (DI template)	E02 (Mistral)	E03 (CU+gpt-5.2)	E04 (gpt-5.4 direct)	E05 (gpt-5.4 hybrid)	E07 (gpt-4o hybrid)	E08 (gpt-5.2 hybrid)	E06 (ensemble)
sin	2	0.825	0.861	0.934	0.809	0.923	0.923	0.950	0.898
date	2	0.907	0.873	0.920	0.693	0.936	0.936	0.909	0.936
phone	2	0.818	0.884	0.988	0.820	0.936	0.909	0.988	0.963
name	2	0.696	0.779	0.805	0.777	0.880	0.870	0.870	0.880
signature	2	0.605	0.579	0.613	0.509	0.675	0.679	0.652	0.680
freeform_text	1	0.575	0.600	0.550	0.575	0.650	0.750	0.650	0.700
checkboxes	28	0.952	0.939	0.996	0.885	0.952	0.941	0.973	0.989

category	n fields	E00 (DI template)	E02 (Mistral)	E03 (CU+gpt-5.2)	E04 (gpt-5.4 direct)	E05 (gpt-5.4 hybrid)	E07 (gpt-4o hybrid)	E08 (gpt-5.2 hybrid)	E06 (ensemble)
income_amounts	37	0.852	0.905	0.881	0.912	0.951	0.884	0.968	0.953

Category leaders among the single engines:

- **E03 (Azure CU + GPT-5.2)** wins checkboxes (0.996) and ties phone (0.988) — CU's dedicated `selectionMark` primitive is purpose-built for box-style yes/no inputs and edges out every VLM-based approach when it works, and CU's phone normalisation is reliable. (Caveat: E03's `selectionMark` has a separate hard-failure mode on a small number of edge-case samples where it flips spouse-column `_no` checkboxes to "selected" — covered in the [E08 deep dive](#).)
- **E08 (gpt-5.2 hybrid)** wins on structured-content categories — `sin` (0.950), `income_amounts` (0.968), and ties phone (0.988). The same gpt-5.2 model that powers E03's generative leg outperforms E03 on these categories when paired with Azure DI prebuilt-layout instead of CU's content-extraction layer. DI's OCR layer transcribes dense digit handwriting more reliably than CU's on this dataset.
- **E05 (gpt-5.4 hybrid)** wins on date (tied with E07 at 0.936) and on name (0.880, tied with E07 / E08). gpt-5.4 on this pipeline is best at interpretive text fields with fewer literal-character constraints.
- **E07 (gpt-4o hybrid)** wins on signature (0.679) and `freeform_text` (0.750) — somewhat surprising. gpt-4o seems to handle the squiggle-vs-handprint distinction on signatures slightly better than the newer models, and pulls more of the `explain_changes` text verbatim.

Where E08 makes up ground on E03 — most of E08's aggregate lead over E03 is on `income_amounts` (+8.7 pp: 0.968 vs 0.881) and `sin` (+1.6 pp: 0.950 vs 0.934). These are categories where literal-digit fidelity matters most. Both engines run the same gpt-5.2 generative model; two architectural choices explain the gap: (a) DI's standalone prebuilt-layout transcribes handwritten digits with less aggressive grounding than CU's prebuilt-layout, so it drops fewer small zeros and faint glyphs; (b) E08's gpt-5.2 receives the image alongside the OCR Markdown, so it can recover digits the OCR layer mistranscribed by reading them off the image — CU's gpt-5.2 never sees the image and can only work with what the OCR produced.

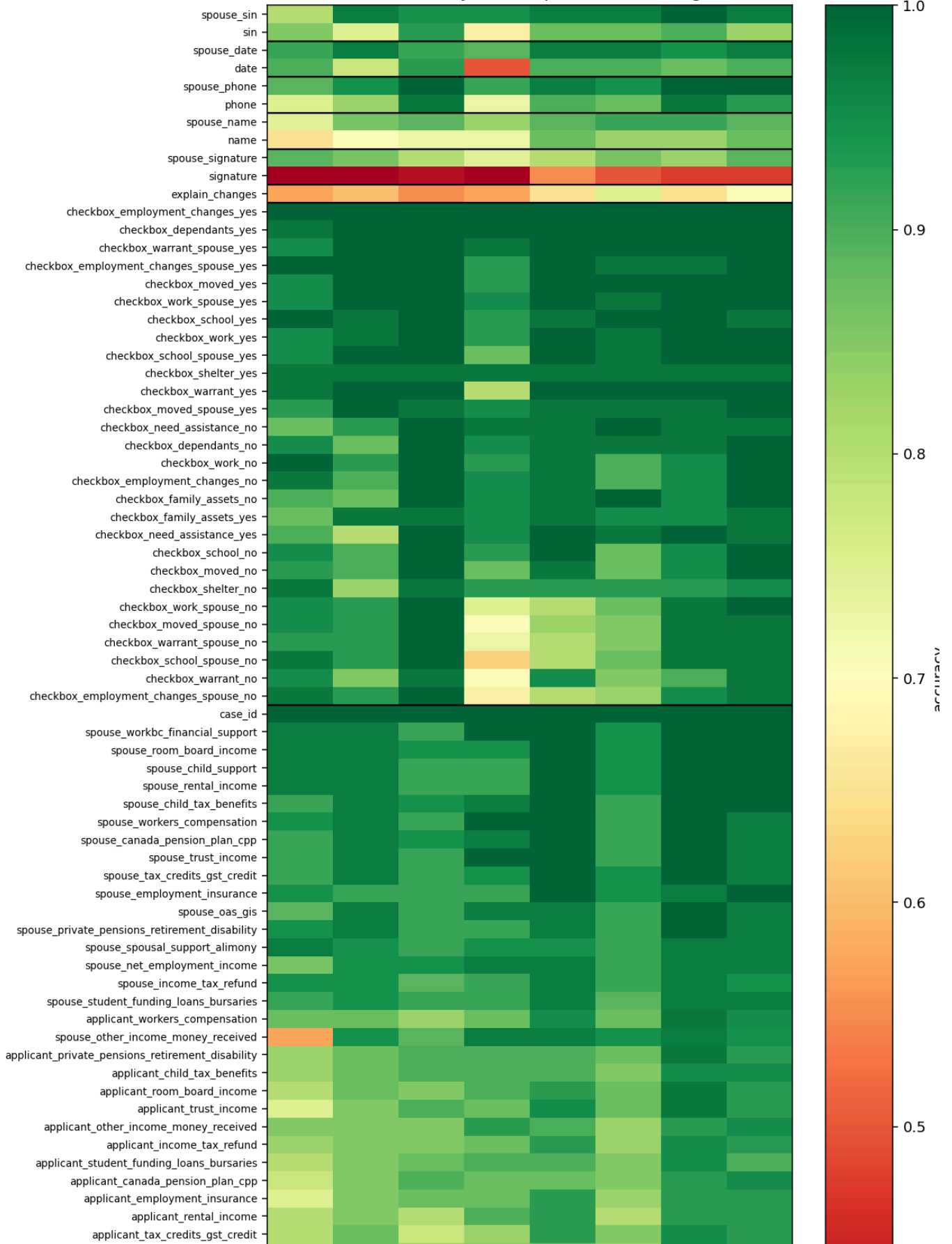
Category floor for every engine: signature and freeform_text (~0.55–0.70 across all engines). These two categories are weakest because the metric isn't a great fit:

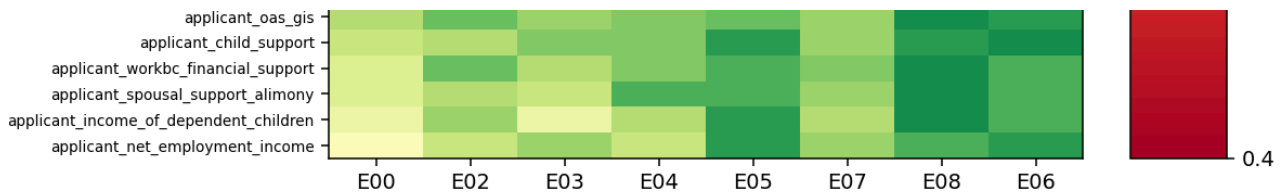
- **Signature** is being scored as exact text matching. But what matters in production is usually whether a signature *exists* on the form, not what the squiggle actually spells out. Re-scoring signature as presence-or-absence would lift every engine substantially.
- `explain_changes` (the freeform text field) is a single long natural-language string per document. Strict equality fails on any minor wording difference, so the metric reads as low even when the engine's extraction is substantively correct. Fuzzy matching (e.g. Levenshtein similarity ≥ 0.85) would be a more honest metric for this field.

So the floor on these two categories isn't an OCR or extraction problem — it's a metric-choice issue. The aggregate F1 numbers would rise noticeably if these two were measured against more appropriate rules.

Per-field accuracy heatmap

Per-field accuracy heatmap (74 fields x engines)





How to read: Each row is one of the 74 SDPR fields, grouped by category (horizontal black lines mark category boundaries) and sorted within each category by mean accuracy. Each column is an engine. Cell colour is the field's accuracy across the 40 samples — red $\approx$ 40% or worse, yellow $\approx$ 70%, green $\approx$ 100%. Visual stripes within a category mean "all 5 engines struggle here in the same way"; lone-red cells mean "this engine has a specific weak spot the others don't".

The heatmap surfaces the long-tail variation that the per-category averages hide:

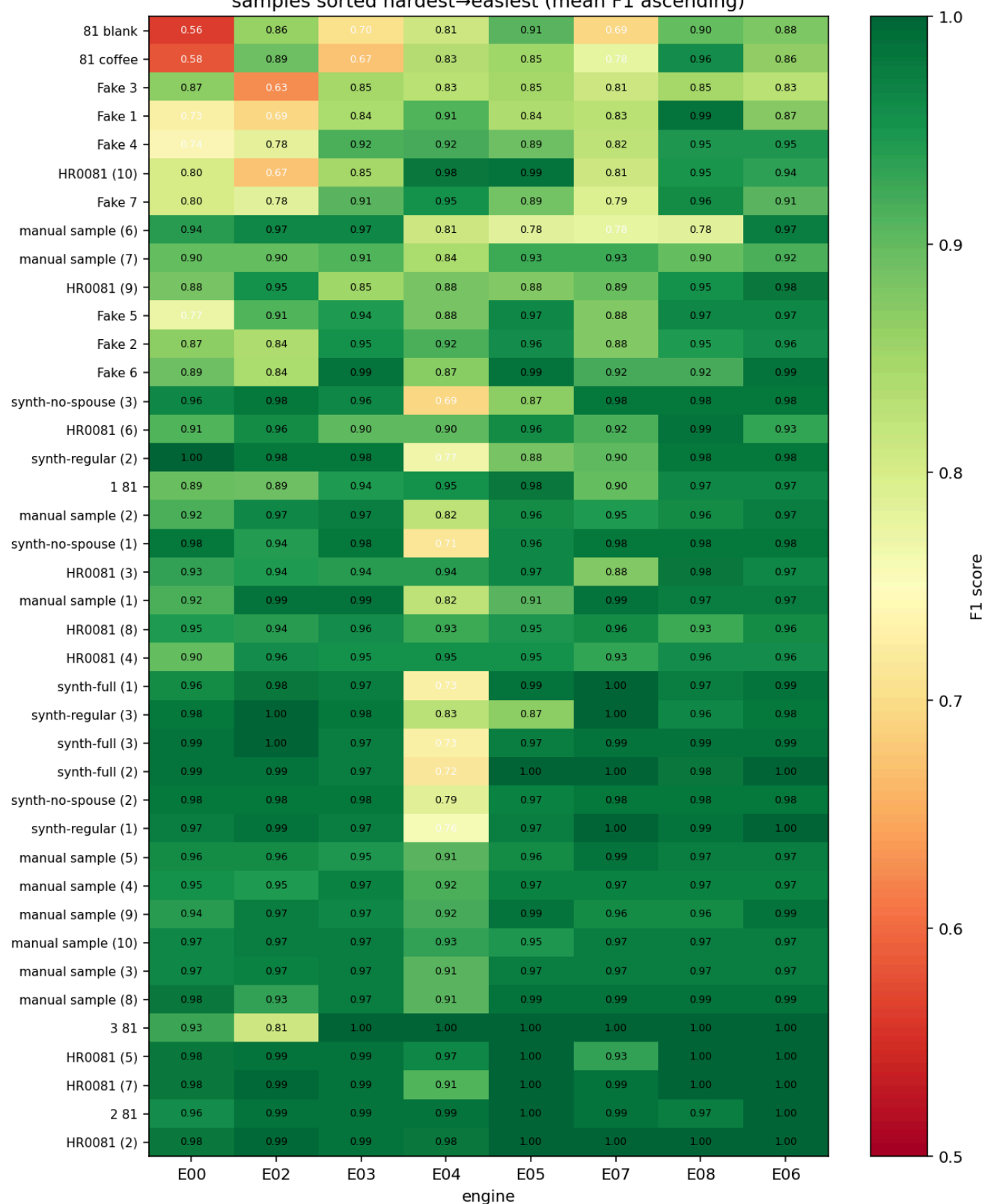
- Within `checkboxes`, the weakest fields are the *spouse-column* "No" boxes (`checkbox_employment_changes_spouse_no`, `checkbox_school_spouse_no`, `checkbox_work_spouse_no`, etc.). The failure mode is **flipping the empty `_no` box to "selected"**, and three engines fall into it on different sample subsets:
 - **E07 (gpt-4o hybrid)** does this broadly — drops to 0.62–0.75 across many samples; the most pervasive instance of the failure.
 - **E04 (gpt-5.4 VLM-direct)** does it at similar rate (0.62–0.75) — same checkbox-rendering interaction with gpt-5.4's vision encoder without an OCR pre-pass.
 - **E03 (CU + gpt-5.2)** does it on a smaller but concentrated set of samples (`81 coffee`, `Fake 1`, etc.) where CU's `selectionMark` primitive misclassifies the obscured/edge-case checkbox.
 - **E08 (gpt-5.2 hybrid)** does it on exactly one sample (`manual sample (6)`), where all `_no` boxes flip at once.
 - **E05 (gpt-5.4 hybrid)** rarely if ever falls into this mode despite using the same gpt-5.4 model as E04 — the OCR pre-pass apparently helps gpt-5.4 anchor on the actual checkbox state more reliably than the image alone.

So the failure is a property of the checkbox-detection layer (CU's `selectionMark` primitive, or the VLM's vision encoder), not the underlying generative model alone. Each engine has its own sample subset where this fires.

- Within `income_amounts`, the weakest fields are the *applicant-column* income lines (`applicant_net_employment_income`, `applicant_income_of_dependent_children`, `applicant_spousal_support_alimony`, etc.), where E00 and E02 sit in the 0.70–0.80 range. The spouse-column equivalents are 5–15 pp easier for the same engines. The pattern across engines suggests applicant-column income cells in this dataset are systematically harder to read than spouse-column ones — possibly because the applicant fields have more variation in handwriting and value range across the samples. E08 (gpt-5.2) handles these more consistently than the other VLMs, contributing to its top per-category score.
- E04's row is visibly redder than the others in the date band — gpt-5.4's vision encoder makes systematic year misreads on hand-written dates regardless of which sample it's looking at. When the same gpt-5.4 model gets an OCR pre-pass (E05), the date band recovers.

Per-sample F1 heatmap

Per-sample F1 heatmap (40 samples × engines)
samples sorted hardest→easiest (mean F1 ascending)



How to read: Each row is one of the 40 samples, sorted from hardest (top) to easiest (bottom) by the mean F1 across engines. Each column is an engine. Cells are coloured by F1 — red ≈ 0.5 or worse, yellow ≈ 0.78, green ≈ 1.0 — and the

numeric F1 is printed in each cell. Rows that are red across the board are the genuinely hard samples (e.g. 81 blank, 81 coffee); rows that are red on one engine but green elsewhere are engine-specific failures.

This view makes the failure clusters obvious:

- **The top 3 rows** (81 blank, Fake 3, 81 coffee) are the dataset's floor. 81 blank and 81 coffee are the intentionally-hard samples (see [Failure-mode samples](#) below). Fake 3 is hard mostly for E02 (F1 0.63 there); the other engines clear 0.83 on this sample.
- **The Fake series** (Fake 1, Fake 4, Fake 5, Fake 7) is where the OCR-based engines E00 and E02 struggle most — they drop to F1 0.69–0.80 on these handwritten samples, while E03/E04/E05 mostly clear 0.85.
- **The synth-* cluster** is the densest set of forms in the dataset (most cells filled). **E04 (GPT-5.4 VLM-direct) specifically fails on this cluster** — every synth sample drops E04 below 0.80 (F1 0.69–0.79). The OCR pre-pass in E05 closes most of the gap on the same gpt-5.4 model; E03 also handles these samples cleanly. The pattern is consistent with GPT-5.4's vision encoder running out of headroom on dense pages — when OCR markdown is added, the model has a literal anchor and the issue largely goes away.
- **HR0081 (10)** is hard specifically for E02 (F1 0.67) — Mistral cannot read its handwriting in the OCR pass. Every other engine clears 0.80 on it.
- **The bottom ~25 samples** are largely green across the board; the engine ranking on the easy samples is essentially noise.

Per-engine deep dive

E00 — Azure DI custom template

Headline: field accuracy **0.872**, F1.median **0.939**, F1.mean **0.903**, precision.mean **0.917**, recall.mean **0.899**, FP.mean **5.60**.

Methodology: a supervised, form-specific model trained inside Azure Document Intelligence. A representative set of forms was uploaded with manual field-position labels and value annotations; Azure trained a custom template model that locates each field by its position on the form and reads the value at that position. At inference time, the engine submits the page image to the trained model, polls the long-running operation until terminal, and receives a JSON object of { field_key: value } pairs. There is no prompt — extraction behaviour is entirely a function of the labels in the training set. This is the same workflow that produced the V1 Report.

Strengths:

- Reasonably balanced — no single category drops below 0.60.
- Highest single-engine accuracy on **date** (0.907) — the labelled training data exposed it to the form's specific date positions.
- Strong on **checkboxes** (0.952) — tied with E05.

Weaknesses:

- **Worst single-engine on name (0.696) and signature (0.605).** The template reads from fixed bounding boxes; handwritten signatures and names vary in placement and the template's geometric tolerance can't always keep up.
- **High FP.mean (5.60)** — substitution errors (engine produces a wrong value rather than returning null) dominate. The template extracts a value even when the cell is empty on some samples.
- **Lowest field accuracy of the five single engines (0.872).** The supervised-template approach has been overtaken by general-purpose generative engines.

Errors by category (raw counts / total predictions): sin 13/75 (17.3%), date 7/75 (9.3%), phone 14/75 (18.7%), name 23/75 (30.7%), signature 31/75 (41.3%), freeform 17/40 (42.5%), checkboxes 54/1120 (4.8%), income 205/1317 (15.6%).

E02 — Mistral on Azure AI Foundry

Headline: field accuracy **0.896**, F1.median **0.959**, F1.mean **0.918**, precision.mean **0.941**, recall.mean **0.902**, FP.mean **4.05**.

Methodology: Mistral's general-purpose Document AI model, accessed via the Azure AI Foundry deployment route. Each document goes through two passes inside Mistral: (1) an OCR pass that transcribes the page to text + markdown, and (2) a generative annotation pass that maps that text into a JSON schema. We supply a JSON Schema of the 74 fields (with per-field descriptions) plus a ~2 KB instruction prompt that describes the SDPR form's two-column income layout, checkbox conventions, and blank-vs-zero rules. Important caveat: **Mistral's annotation pass on this deployment reads only the OCR text output from the first pass, not the raw image** — so anything the OCR layer fails to transcribe (small or sloppy handwriting) cannot be recovered by the structured pass.

Strengths:

- **Highest precision among the OCR-based engines (0.941)** — Mistral is generally conservative; when it doesn't see a field clearly, it tends to return null rather than guessing.
- F1.median (0.959) is competitive with the gpt-5.4-based engines.

Weaknesses:

- **Worst single-engine on signature (0.579)**, second-worst on name (0.779) — the OCR-only annotation pipeline can't disambiguate handwritten "X" the signature from "X" the unselected-checkbox glyph.
- **Lowest performance on the hand-written Fake series and HR0081 (10)** — F1 0.626 to 0.785 on these samples. The pattern points back to Mistral's structural limitation: the annotation pass sees only what the OCR layer transcribed, so handwriting that doesn't OCR cleanly is lost.
- **14 blank-when-zero errors** (predicted null where GT is 0) — Mistral interprets blank-looking cells conservatively and under-extracts the literal zero. See [the blank-vs-zero problem](#).

Errors by category: sin 11/75 (14.7%), date 10/75 (13.3%), phone 9/75 (12.0%), name 17/75 (22.7%), signature 33/75 (44.0%), freeform 16/40 (40.0%), checkboxes 68/1120 (6.1%), income 133/1317 (10.1%).

E03 — Azure Content Understanding + GPT-5.2

Headline: field accuracy **0.914**, F1.median **0.969**, F1.mean **0.938**, precision.mean **0.957**, recall.mean **0.923**, FP.mean **2.88**.

Methodology: Azure Content Understanding is a managed two-stage extraction service. Per Microsoft's GA documentation (November 2025):

1. **Stage 1 — Content Extraction.** A specialized OCR/Layout model from the Document Intelligence family (Microsoft Learn: *"prebuilt-read and prebuilt-layout analyzers now bring key Document Intelligence capabilities to Content Understanding"*) reads the image and produces a layout-aware Markdown representation plus parallel structural metadata (pages, paragraphs, sections, tables with HTML-like markup, words with bounding polygons, selection-mark detector output, confidence scores). Crucially this stage is **tuned for stricter grounding than the standalone DI prebuilt-layout model** — Microsoft's own Q&A documents cases where CU's prebuilt returns empty content for documents the standalone DI prebuilt parses cleanly: *"Content Understanding prebuilt analyzers are designed for higher-level content extraction, RAG, and domain-specific scenarios, not as a drop-in replacement for raw OCR."* No LLM is involved at this stage.
2. **Stage 2 — Field Extraction.** A customer-deployed Foundry GPT-5.2 chat model receives the **Markdown output from stage 1** plus the analyzer's JSON schema (field names, typed property declarations, per-field descriptions) and returns a structured JSON object. **The image is not sent to the LLM** — stage 2 is text-only at the model layer. Microsoft handles the prompt construction (the "Contextualization" layer), grounding back-mapping, confidence scoring, and output normalization.

Our analyzer configuration includes:

- One typed field per output key (string, number, date) with a per-field description string.
- Checkbox fields are mapped to CU's `selectionMark` primitive (`classify` method with enum `selected` / `unselected`), which runs as a dedicated specialized model in stage 1.

- A global ~2 KB instruction string describing the form's column conventions, blank-vs-zero rules, and checkbox semantics — sent as part of the analyzer schema; how exactly CU's Contextualization layer surfaces this to the LLM is hidden.

CU "forces grounding" (Microsoft Q&A): *"Content Understanding forces grounding — anchoring outputs in the text of the input documents — and will not return answers if they cannot be grounded."* So if the stage-1 OCR layer didn't transcribe a value, stage 2 returns null rather than guessing from context.

Strengths:

- **Best single engine on checkboxes (0.996) and phone (0.988, tied with E08)** — CU's dedicated `selectionMark` primitive is purpose-built for box-style inputs and produces consistent phone normalisation.
- **Low FP.mean (2.88)** — second-lowest among single engines after E08.

Weaknesses:

- **Underperforms E08 even though both run gpt-5.2.** Two architectural differences drive the 2.2 pp F1.mean gap: (1) CU's stage-1 OCR is tuned for stricter grounding than the standalone DI prebuilt-layout that E08 uses — it drops more handwritten zeros and faint characters by design; and (2) CU's stage 2 is text-only — the LLM never sees the image — while E08's chat call includes the image alongside the OCR Markdown with an instruction to trust the image when they disagree. See the [E08 deep dive](#) for the full mechanism breakdown.
- **95 blank-when-zero errors** — CU's OCR layer drops small handwritten zeros at high rate. Worse than every engine in the report except E07 (gpt-4o hybrid). See [the blank-vs-zero problem](#).
- **165 income-amount mismatches (12.5%)** — the dominant error category. The OCR layer's digit transcription is the bottleneck; the generative pass gets reliable schema + instructions but cannot recover digits the OCR layer mistranscribed (because by architecture, the LLM only sees the Markdown — it has no image to fall back on).
- **Signature (0.613) and freeform_text (0.550)** sit in the lower tier — CU treats signature as a generic string field; the metric is also a poor fit for these two categories (see [Per-category field accuracy](#)).

Errors by category: sin 5/75 (6.7%), date 6/75 (8.0%), phone 1/75 (1.3%), name 15/75 (20.0%), signature 30/75 (40.0%), freeform 18/40 (45.0%), checkboxes 5/1120 (0.4%), income 165/1317 (12.5%).

E04 — GPT-5.4 vision-language model (direct)

Headline: field accuracy **0.868**, F1.median **0.903**, F1.mean **0.870**, precision.mean **0.876** (lowest), recall.mean **0.866**, FP.mean **8.48** (highest) — the weakest engine overall.

Methodology: A direct call to Azure OpenAI's GPT-5.4 deployment with the form page sent as an inline base64-encoded image. The request uses strict JSON-Schema response formatting — GPT-5.4 must return a JSON object that conforms to a schema with one property per field key, plus a sibling `source_quotes` object mapping each field to a verbatim quote the model believes it pulled the value from. The same ~2.5 KB instruction prompt as the other generative engines describes the form layout, column conventions, and blank-vs-zero rules. There is **no OCR pre-pass** — GPT-5.4's vision encoder reads the image directly.

Strengths:

- Strong on `income_amounts` (0.912) **at the dataset level**, but see the failure pattern below.
- Fastest wallclock per sample (~5.9 s), no separate OCR step.

Weaknesses:

- **Lowest precision (0.876) and highest FP.mean (8.48)** — when GPT-5.4 vision-direct doesn't know a value, it tends to fill in a plausible-looking wrong value rather than returning null.
- *Failure pattern concentrates on the dense synth- samples** — every synth sample drops E04 below F1 0.80 (F1 0.69–0.79, with 11–21 false-positive predictions per sample). These are the most densely-filled documents in the dataset; GPT-5.4's vision encoder appears to struggle as soon as the page is packed with digit-heavy handwritten content. Less-dense samples don't reproduce this — even other handwritten forms like `HR0081 (10)`, where many cells are left

blank, sit at F1 0.98 on E04. The OCR pre-pass in E05 (same gpt-5.4 model, just with DI markdown added) closes most of the synth gap, so the failure is the vision encoder running out of headroom on dense images rather than a deeper reasoning issue.

- **Worst single-engine on date (0.693)** — the GPT-5.4 vision encoder makes systematic year misreads on hand-written dates (e.g. 2023 for 2025, 2020 for 2026). These are real digit misreads, not format variants.
- **Worst single-engine on signature (0.509) and checkboxes (0.885)** — the latter 9 pp below E03; GPT-5.4 doesn't distinguish marked from unmarked checkboxes as reliably as CU's structured primitive does.

The pattern is consistent: **E04 trades structural reliability for the simplicity of a single VLM call**. When the same GPT-5.4 model is given an OCR pre-pass (E05), F1.mean jumps from 0.870 to 0.942 and FP.mean drops from 8.48 to 3.38 — that gap is the concrete value of an OCR layer in front of a VLM for schema-driven extraction.

Errors by category: sin 15/75 (20.0%), date 24/75 (32.0%), phone 14/75 (18.7%), name 17/75 (22.7%), signature 38/75 (50.7%), freeform 17/40 (42.5%), checkboxes 129/1120 (11.5%), income 123/1317 (9.3%).

E05 — GPT-5.4 VLM + Azure DI layout (hybrid)

Headline: field accuracy **0.934**, F1.median **0.960**, F1.mean **0.942**, precision.mean **0.951**, recall.mean **0.935**, FP.mean **3.38**, matchedFields.median **71** (highest among single engines).

Methodology: A two-pass setup that combines the strengths of OCR and a vision LLM:

1. Azure Document Intelligence's `prebuilt-layout` reader transcribes the page to markdown plus per-word bounding boxes.
2. GPT-5.4 receives **both** the raw image **and** the OCR markdown, wrapped in `<ocr_text>` delimiters. The system prompt explicitly tells the model: *"Use both inputs together. The OCR text helps you locate fields and read structure. The image is the source of truth. When the OCR text and the image disagree, trust the image and ignore the OCR text."*

The same ~2.5 KB instruction prompt as E03/E04 describes the form's column conventions, blank-vs-zero rules, and checkbox semantics. The output schema is identical to E04's (one property per field plus a sibling `source_quotes` mapping).

Strengths:

- **Best single-engine on 5 of 8 categories:** sin (0.923), date (0.936), name (0.880), signature (0.675), income_amounts (0.951).
- **Zero blank-when-zero errors** — uniquely among the engines, the hybrid never under-extracts a literal zero. The DI OCR pre-pass transcribes small `0` glyphs that the standalone CU and Mistral OCR layers miss, and GPT-5.4 trusts the OCR text on numerals.
- **Highest matched-fields median (71)** — closer to the 74-field ceiling than any other engine.
- Carries real word/line bounding boxes through to the final output, which downstream consumers (cleanup, confidence checking) can use.

Weaknesses:

- `checkboxes` (0.952) tied with E00, below E03's 0.975 — the hybrid's checkbox accuracy is constrained by GPT-5.4's vision, not by the OCR layer. CU's analyzer schema with dedicated checkbox primitive is still better for this category.
- `phone` (0.936) below E03 (0.963) — DI's OCR markdown drops parentheses around area codes; GPT-5.4 trusts that and the variant doesn't match the GT exactly.
- Slight over-extraction tendency on a small number of hand-written samples (FP.mean 3.38 vs E03's 3.00).

Errors by category: sin 6/75 (8.0%), date 5/75 (6.7%), phone 5/75 (6.7%), name 9/75 (12.0%), signature 25/75 (33.3%), freeform 14/40 (35.0%), checkboxes 54/1120 (4.8%), income 70/1317 (5.3%).

E07 — GPT-4o VLM + Azure DI layout (hybrid)

Headline: field accuracy **0.899**, F1.median **0.952**, F1.mean **0.923**, precision.mean **0.942**, recall.mean **0.909**, FP.mean **4.00**, matchedFields.median **68**.

Methodology: Identical pipeline to E05 — the only change is the generative model. Azure DI prebuilt-layout transcribes the page to markdown plus per-word bounding boxes; the GPT-4o deployment receives both the raw image and the OCR markdown (wrapped in `<ocr_text>` delimiters); the response is constrained by the same strict JSON-Schema response format with the same prompt and field descriptions as E05.

Strengths:

- **Best single-engine on signature (0.679) and freeform_text (0.750)** — gpt-4o pulls these text categories slightly more verbatim than gpt-5.4 or gpt-5.2.
- Fastest wallclock among the hybrid trio (~326 s vs E05's 344 s and E08's 356 s).
- Cost: lowest per-token rate of the three VLMs.

Weaknesses:

- **Largest single failure mode of any engine: the spouse-column `_no` checkboxes flip to "selected" on empty cells.** Visible as a contiguous red band on the heatmap — five consecutive checkbox fields (`checkbox_employment_changes_spouse_no`, `checkbox_school_spouse_no`, `checkbox_work_spouse_no`, `checkbox_moved_spouse_no`, `checkbox_warrant_spouse_no`) all drop to 0.62–0.75 on E07 while holding ≥ 0.93 on E08. This single pattern accounts for ~half the F1 gap vs E08 and explains five of the seven worst regression samples (`81 blank`, `Fake 5/7`, `HR0081 (3)`, `1 81`).
- **Worst single-sample F1 of any single engine on `81 blank` (0.692)** — a 21 pp drop vs E05's 0.905 on the same sample. The checkbox failure mode dominates here (10 of E07's 27 false positives on this sample come from the spouse `_no` boxes).
- Trails E05 by ~2 pp F1.mean and E08 by ~3.7 pp. The model gap shows up as both higher FP rate and lower recall.

The pattern is consistent: **gpt-4o is the weakest of the three VLMs on this task at this date**. Production-wise, the cost savings vs gpt-5.4 / gpt-5.2 do not appear to justify the accuracy drop on this dataset.

Errors by category: sin 6/75 (8.0%), date 5/75 (6.7%), phone 7/75 (9.3%), name 10/75 (13.3%), signature 25/75 (33.3%), freeform 10/40 (25.0%), checkboxes 66/1120 (5.9%), income 153/1317 (11.6%).

E08 — GPT-5.2 VLM + Azure DI layout (hybrid)

Headline: field accuracy **0.952**, F1.median **0.973**, F1.mean **0.960**, precision.mean **0.965**, recall.mean **0.955**, FP.mean **2.50**, matchedFields.median **71** — **the strongest single engine overall**.

Methodology: Same pipeline as E05 and E07 — Azure DI prebuilt-layout → image + OCR markdown → strict-mode JSON Schema. The generative model is GPT-5.2 (already deployed for E03's Content Understanding generative leg, so no new Azure infrastructure was needed).

Strengths:

- **Best single engine on every accuracy aggregate.** Beats E03 by 2.2 pp F1.mean even though both engines run the same gpt-5.2 generative model. The mechanism breakdown is in the "Why E08 beats E03 despite both using gpt-5.2" paragraph below.
- **Best single-engine on sin (0.950), phone (0.988), and income_amounts (0.968)** — the structured-digit categories where literal-character fidelity matters most.
- **Tightest per-sample F1 distribution of any engine** (stdDev 0.042; 28 of 40 samples sit in the 0.95–0.99 band, no samples below 0.78). This consistency is the main practical edge over E03 / E05.
- **Best single-engine FP.mean (2.50)** — lowest wrong-value substitution rate.
- **Most dramatic single-sample improvement over E05:** `81 coffee` (a historically-floor obscured-form sample) jumps from F1 0.851 → 0.959. `Fake 1` (a pencil-filled form) jumps from 0.837 → 0.986.

Weaknesses:

- **One residual hard sample (manual sample (6) at F1 0.784)** — the same sample that E05 lands lowest on. Dense handwritten income figures with ambiguous digit shapes; neither gpt-5.2 nor gpt-5.4 reads it cleanly.
- Slightly behind E05 on date (0.909 vs 0.936) — a 3-sample window where gpt-5.4 read a hand-written month-day cleanly but gpt-5.2 didn't.

Why E08 beats E03 despite both using gpt-5.2. Microsoft's GA documentation (November 2025) is unambiguous about CU's architecture: Content Understanding is a **two-stage pipeline that pairs Microsoft's Document Intelligence OCR/Layout models with a customer-supplied Foundry LLM deployment**. Step 1 is `prebuilt-layout` (the same model E08 uses directly), step 2 is gpt-5.2 in this configuration. So on paper, E03 and E08 are running the same model family in the same order. Yet E08 wins by 2.2 pp F1.mean, 3.2 pp recall, and 0.4 lower FP.mean — and the gap is much larger than measurement noise. Several concrete architectural differences explain it:

1. **CU's `prebuilt-layout` is tuned for grounding-strict behaviour, not maximum OCR recall.** Microsoft's own Q&A documents real cases where CU's `prebuilt-layout` returns *empty content* for a PDF that Document Intelligence's standalone `prebuilt-layout` parses cleanly — Microsoft confirms this is *by design*: CU's prebuilts are tuned for "higher-level content extraction, RAG, and domain-specific scenarios," not as a drop-in replacement for raw OCR. Practical effect on this dataset: CU's OCR drops small handwritten zeros and faint marginal characters that standalone DI `prebuilt-layout` transcribes. E08 dropped 7 zeros across the dataset; E03 dropped 95. The income-amounts category — where most of the small `0`s live — accounts for the bulk of the gap (E08 0.968 vs E03 0.881, +8.7 pp).
2. **CU never sends the document image to the LLM.** Per Microsoft's overview, CU's stage 2 receives "the layout-aware Markdown produced by step 1" plus the analyzer's schema — *not* the raw image. CU is a text-and-Markdown GenAI pipeline at the LLM layer, regardless of whether the input was an image or PDF. **E08 sends both the image AND the OCR Markdown to gpt-5.2**, with an explicit instruction in the system prompt that *"the image is the source of truth — when the OCR text and the image disagree, trust the image."* This is the dominant mechanism behind E08's recall advantage: when DI's `prebuilt-layout` drops a character or misreads a digit in E08, gpt-5.2 can still see the image and recover the correct value. When CU's stricter `prebuilt-layout` drops content, the LLM has nothing to recover from — by architectural construction, it never saw the pixels.
3. **CU's prompt is hidden and schema-driven; ours is direct and prompt-driven.** Per Microsoft's docs, CU's stage 2 prompt is built by a managed "Contextualization" layer from the analyzer schema (field names, types, descriptions, optional in-context examples) — *not* from a customer-written instruction string. We never see the actual prompt CU's gpt-5.2 receives. In E08 we control the system prompt directly: ~2.5 KB of explicit instructions about column conventions, blank-vs-zero rules, signature-vs-name distinctions, and an instruction to return null when uncertain. Two concrete consequences:
 - **CU's gpt-5.2 hallucinates plausible-but-wrong values when the OCR is uncertain.** On HR0081 (6) E03 returns `signature: "Junal"` (GT Trung), `spouse_name: "April Lune"` (GT April Luna), `spouse_signature: "Aku De"` (GT AKM De). These are fluent guesses, not character-level OCR errors — the model is filling cells from imagination. E08 on the same sample returns the correct values because the prompt explicitly says don't guess and the image is available to verify.
 - **CU's `selectionMark` primitive misclassifies obscured checkboxes** on 81 coffee, Fake 1, etc. — flipping empty spouse-column `_no` boxes to `"selected"`. This is one of CU's "specialized models" running in stage 1 (Microsoft uses dedicated selection-mark detectors before the LLM ever sees the document). When the dedicated detector misclassifies, the wrong value is what reaches the LLM, and the LLM has no image to cross-check against. E08 has its own checkbox failure on exactly one sample (manual sample (6)) — one sample's worth of damage vs CU's spread across several samples.
4. **CU "forces grounding" — it returns empty values when uncertain.** Microsoft's Q&A documents this explicitly: *"Content Understanding forces grounding — anchoring outputs in the text of the input documents — and will not return answers if they cannot be grounded."* This is good for precision but costs recall: when the answer is on the form but the OCR layer didn't capture it cleanly, CU returns null. The 95 dropped zeros are largely instances of this — the `0` glyph is on the form, the OCR layer didn't capture it, so the LLM cannot ground an answer and CU returns blank.

What this means architecturally

So E08 is *not* a thin re-implementation of CU. The pipelines diverge on two architectural axes — **the LLM sees the image in E08 but not in E03**, and **the OCR layer in E03 is tuned for stricter grounding than the standalone DI prebuilt-layout that E08 uses**. Both of these favour recall in E08 at the cost of stricter grounding in E03.

Cost trade-off — important caveat

E08 is meaningfully more expensive per call than E03. The two pipelines pay roughly the same for the OCR layer (DI prebuilt-layout / CU content-extraction are similar-priced per page). But the LLM stage differs:

- **E03 (CU)** sends only the layout Markdown to gpt-5.2. Markdown for one SDPR form is ~4–5 KB, on the order of ~1,000–2,000 input tokens to the LLM, plus CU charges a fixed "contextualization" fee of ~1,000 tokens per page on top of the customer's own Foundry-deployment token bill.
- **E08 (custom)** sends the same OCR Markdown *plus* the document image as an inline base64-encoded image attachment. The image alone is ~1,500–2,500 vision tokens for a single-page SDPR form at 200 DPI — on top of the ~1,000–2,000 text tokens. Net E08 input-token cost per call is **roughly 2–3× E03's LLM-stage input tokens** (and image tokens are billed at the same per-token rate as text on gpt-5.x Foundry deployments).

So E08 buys its accuracy lead by spending more on inference per page. The trade is real and quantifiable: ~2.2 pp F1.mean improvement for ~2–3× LLM input-token cost on stage 2. For workloads where every percent of accuracy matters (regulated extractions, billing data, compliance), E08 is the better engine. For workloads where the form is well-structured and OCR-friendly, CU's grounding-strict, cheaper, image-free path may be the better fit.

Net interpretation

"Same generative model" doesn't mean "same engine". CU and E08 share *most* of their architecture — same OCR family, same generative model, similar two-stage pipeline. But two architectural choices (image goes to the LLM in E08 only; CU's OCR is tuned stricter than standalone DI) drive a meaningful recall and FP-rate advantage for E08 on this dataset, at the cost of higher per-call inference. The result is consistent with what Microsoft's own GA documentation suggests: CU trades flexibility and recall for managed grounding/confidence/multimodality and a stricter quality floor. We've intentionally traded the other way.

Errors by category: sin 4/75 (5.3%), date 7/75 (9.3%), phone 2/75 (2.7%), name 9/75 (12.0%), signature 26/75 (34.7%), freeform 14/40 (35.0%), checkboxes 30/1120 (2.7%), income 42/1317 (3.2%).

Same-pipeline model bake-off (E05 / E07 / E08)

E05, E07, and E08 run an identical pipeline with three different generative models. With the pipeline / prompt / field descriptions held constant, the gap between them is entirely attributable to the VLM. The ordering on this dataset:

metric	E08 (gpt-5.2)	E05 (gpt-5.4)	E07 (gpt-4o)
f1.mean	0.960	0.942	0.923
f1.median	0.973	0.961	0.952
precision.mean	0.965	0.951	0.942
recall.mean	0.955	0.935	0.909
matchedFields.median	71	71	68
falsePositives.mean	2.50	3.38	4.00

metric	E08 (gpt-5.2)	E05 (gpt-5.4)	E07 (gpt-4o)
Wallclock	356 s	344 s	326 s

Counter-intuitive direction: the older model number wins. gpt-5.2 is roughly 1.8 pp F1.mean ahead of the more recently released gpt-5.4 on this task. The most likely explanation is workload tuning — gpt-5.2 is the model Microsoft chose for Content Understanding's generative layer, which is essentially the same shape of structured form extraction as this benchmark. gpt-5.4 generalises better elsewhere, but loses on the specific extraction format here. gpt-4o sits clearly behind both.

The gap between E05 (gpt-5.4) and E07 (gpt-4o) is roughly the same size as the gap between E08 (gpt-5.2) and E05 (gpt-5.4), so model-quality differences on this task scale linearly: 1.5=2 pp F1.mean per generation step. The wallclock differences are small (9% spread across the three).

The blank-vs-zero problem

Across the dataset, **the most common error mode is "engine returns blank when the form has a 0"** — what we call a *blank-when-zero* error. The income-amount fields are particularly affected: many cells on the form are visually empty *and* the GT marks them as `null`, but a few cells contain a hand-written `0` (typically a small loop). Engines vary widely in how often they recognise that small `0` as a zero vs treat it as empty.

engine	blank_when_zero (predicted blank/null, GT 0)	zero_when_blank (predicted 0, GT blank)
E00 (DI custom template)	93	1
E02 (Mistral / Foundry)	96	1
E03 (CU + gpt-5.2)	95	1
E04 (gpt-5.4 VLM-direct)	10	1
E05 (gpt-5.4 hybrid)	29	2
E07 (gpt-4o hybrid)	103	17
E08 (gpt-5.2 hybrid)	7	2

Counts are total per-cell occurrences across the 40-sample dataset (each sample has ~10 GT- `0` numeric cells on the income-amount fields), measured against the current `benchmark-run.json` exports. The previous version of this table reported lower numbers under an earlier evaluator schema; the current numbers come from the standard FP/FN definition adopted on `improve/03`.

E08 (gpt-5.2 hybrid) is the best of any single engine on this metric — only 7 dropped zeros across the whole dataset. **E04 (gpt-5.4 VLM-direct) is second at 10**, surprisingly strong here. The VLM-direct path benefits from gpt-5.4's vision encoder seeing the small handwritten `0` glyphs directly without an OCR layer that might drop them.

E07 (gpt-4o hybrid) is by far the worst — 103 dropped zeros and 17 spurious zeros (10× any other engine on the `zero_when_blank` side). gpt-4o's vision encoder both misses small `0` glyphs more often AND fills in `0` on visually-empty cells that have stray marks. This is the same failure surface that hammers E07 on the spouse-column `_no` checkboxes — gpt-4o is more eager to "see" content where the form is actually blank.

E03 (CU + gpt-5.2) is nearly as bad as E07 and E00 / E02 — 95 dropped zeros — despite using the same generative model as E08. Two compounding effects from CU's architecture: (a) CU's `prebuilt-layout` is tuned for stricter grounding than the standalone DI `prebuilt-layout`, so it drops small handwritten zeros at high rate by design; (b) CU never shows the document image to the generative model, so when the OCR drops a glyph the LLM has no fallback path to recover it. E08 has neither problem — DI's standalone `prebuilt-layout` drops fewer zeros to start with, and the gpt-5.2 call gets the image, so even when

DI does drop a glyph the model can read it directly. E05 (gpt-5.4 hybrid) sits in the middle (29) on the same pipeline as E08 — the difference here is purely the model: gpt-5.4 is slightly more conservative on visual `0` glyphs than gpt-5.2.

The pattern is now clear: **on this metric, the combined effect of "stricter-tuned OCR" + "no image at the LLM" dominates**. E03 vs E08 both use gpt-5.2, but E03 sits at 95 dropped zeros and E08 sits at 7 — and the 13× gap is structural, not model-driven. For production workloads where literal zeros matter (income reporting, financial-document extraction), **E08 is the strongest engine on this dataset** — at the cost of higher per-call inference (image tokens are not cheap).

Failure-mode samples

A handful of samples are worth calling out by name — these are the rows at the top of the [Per-sample F1 heatmap](#), sorted hardest to easiest:

- `81 blank` — an intentionally blank form. F1 ranges from 0.56 (E00) to 0.91 (E05/E08) across engines. The engines fill in some fields that aren't really there (insertions against the mostly-null GT). **E07 (gpt-4o hybrid) is the new worst case here at 0.692** — its empty-checkbox failure mode produces extra false positives on the blank form, dropping it 21 pp below E05 on the same sample. E08 (gpt-5.2) holds 0.898 — essentially tied with E05.
- `81 coffee` — a printed form with coffee stains over the key data. F1 ranges from 0.58 (E00) to **0.959 (E08)**. E08 is the best single engine here; gpt-5.2 reads the unobscured-but-faint cells well. E07 lags at 0.779. E00 fails hardest because its template-position approach can't recover when the data isn't where the template expects.
- `Fake 1` — a form filled out **in pencil**, not pen. Light grey strokes on white paper. F1 ranges from 0.69 (E02) to **0.986 (E08)** — E08 is the best, with E03 and E04 close behind at 0.91. Pencil contrast is hardest for the OCR-first paths (E00, E02), which depend on text-recognition fidelity; the VLMs handle it well.
- `Fake 4` — a phone-camera photo of a form, with visible background around the form edges. The form itself is sharp. F1 ranges from 0.74 (E00) to 0.95 (E06/E08). E00 and E02 substitute heavily on this sample; the VLM-based engines handle the background better. E07 lags the other VLMs at 0.824.
- `Fake 5 / Fake 7` — additional hand-written samples in the next failure tier. **E07 (gpt-4o) sits noticeably lower than E05/E08 here** — Fake 5 drops to 0.877 on E07 vs 0.966 on both E05 and E08; Fake 7 drops to 0.791 on E07 vs 0.887 (E05) / 0.959 (E08). The gpt-4o checkbox failure mode dominates both samples.
- `synth-* cluster (synth-full / synth-no-spouse / synth-regular)` — synthetically-generated forms with hand-written field values. This cluster is where **E04 (GPT-5.4 VLM-direct) underperforms specifically** — every synth sample drops E04 below F1 0.80 (range 0.69–0.79). All other engines, including the hybrid E05/E07/E08 which use the same OCR pre-pass with different VLMs, handle these samples cleanly. **E08 hits F1 0.97–1.00 on every synth sample**, the best of any single engine on this cluster.
- `HR0081 (10)` — a real hand-written form. F1 dropped most on E02 (0.67) — Mistral's pipeline cannot read this sample's handwriting in its OCR pass. **E07 (gpt-4o) also struggles here at 0.810**, a steep drop from E05's 0.986 on the same sample — gpt-4o misreads the dense handwriting where gpt-5.4 reads it cleanly. E03 / E05 / E08 all clear F1 ≥ 0.95 .
- `manual sample (6)` — the lowest-scoring sample on both E05 and E08 (F1 0.784 on both). Dense handwritten income figures with ambiguous digit shapes — the residual hardest sample of the dataset for the VLM hybrid stack. E03 (CU) reads it materially better at F1 0.97.

The first two are the dataset's universal floor — every engine struggles, and these would benefit from human-in-the-loop review by design. The rest are engine-specific failure patterns: pencil contrast hurts the OCR-based engines most; the phone-photo / background sample challenges everyone moderately; the synth-* cluster is specifically a GPT-5.4-vision-direct problem; dense or sloppy handwriting that the OCR layer can't read is specifically Mistral's problem; and the empty `_no` checkbox flip is specifically a gpt-4o problem. **The choice of engine for a production workload should weigh which of these failure modes is most likely in the expected input mix.**

Reflection

1. **E08 (gpt-5.2 hybrid) is the new single-engine winner on every accuracy aggregate.** F1.mean 0.960, F1.median 0.973, precision.mean 0.965, recall.mean 0.955, FP.mean 2.50. It beats E03 — which runs the same gpt-5.2 model internally — by 2.2 pp F1.mean and 3.2 pp recall. The gap is structural, not model-driven: CU's stage-1 OCR is tuned for stricter grounding than the standalone DI prebuilt-layout E08 uses (so it drops 13× more small zeros: 95 vs 7), and CU's stage 2 is text-only — the LLM never sees the image, so it cannot recover content the OCR dropped. E08 sends image + OCR Markdown to gpt-5.2 with an instruction to trust the image. **The accuracy lead is bought at higher per-call inference cost** (image tokens add ~2–3× to the LLM input cost; the OCR-layer cost is similar).
2. **The same-pipeline model bake-off (E05 / E07 / E08) settles a clear ranking on this task: gpt-5.2 > gpt-5.4 > gpt-4o.** With pipeline, prompt, field descriptions, and OCR pre-pass held constant, the gap is entirely attributable to the generative model: E08 wins F1.mean by 1.8 pp over E05 and 3.7 pp over E07. The older-numbered model winning is counter-intuitive but consistent — gpt-5.2 is the same model Microsoft chose for Content Understanding's generative layer, so it has been tuned for exactly this shape of structured form extraction.
3. **Three architectural levers are co-equal in importance; "same model" doesn't mean "same engine".** The E03 vs E08 comparison isolates three differences while holding the generative model (gpt-5.2) constant: (a) OCR tuning — CU's `prebuilt-layout` is tuned for stricter grounding than the standalone DI `prebuilt-layout`; (b) image visibility — CU never shows the image to the LLM, E08 always does; (c) prompt control — CU's prompt is hidden behind its Contextualization layer, E08's is direct. Together these three give a 2.2 pp F1.mean gap. The model-only lever (E05 vs E08, same pipeline different generative model) gives a 1.8 pp gap. Practical implication: when picking a production engine, "we're using gpt-5.2" is far less informative than "we're using DI prebuilt-layout + gpt-5.2 with image-on-the-prompt and a 2.5 KB instruction string". Microsoft's own GA documentation acknowledges this trade-off — CU is designed for managed grounding and confidence; a custom pipeline like E08 trades that for flexibility and recall.
4. **Generative engines + good prompts have eclipsed the custom-trained template** on this form. E00 (a trained Azure DI template — the V1 Report approach) lands at field accuracy 0.872, F1.mean 0.903 — competitive but no longer leading. The four hybrid / generative paths (E02, E03, E05, E08) all sit between 0.918 and 0.960 on F1.mean. The template's structural advantage (it knows the form layout exactly) is matched and exceeded by generative engines once they have field-level descriptions and a workflow-level prompt.
5. **VLM-direct (E04) and gpt-4o (E07) both have precision problems on this workload.** E04 precision 0.876 (FP.mean 8.48) is the worst overall; E07 precision 0.942 (FP.mean 4.00) is the second-worst among the generative engines. Both share a tendency to fill in plausible-but-wrong values on cells where the model isn't certain. The OCR pre-pass closes most of the gap for gpt-5.4 (E04 → E05 jumps F1.mean from 0.870 to 0.942), but gpt-4o's empty-checkbox failure mode persists even with the OCR layer in front. The OCR layer is the bigger lever for *gpt-5.4*; for *gpt-4o*, model choice itself is the lever.
6. **Mistral has a structural ceiling on this dataset.** E02's annotation step reads only the OCR output of its first pass, not the raw image — so when the OCR layer fails to transcribe handwriting cleanly, the structured pass has nothing to work with. This is visible in the `Fake` and `HR0081 (10)` failure cluster, where Mistral lags significantly behind the engines whose generative pass sees the image directly (E03, E05, E08).
7. **Custom-trained models carry a lifecycle cost penalty that generative engines don't.** Even where E00 matches the generative engines on accuracy, the trained template needs re-labelling and re-training whenever the form schema changes. Schema changes on a generative engine are a prompt edit. For forms that evolve over time, the generative path wins on maintenance even when accuracy is roughly equal.
8. **The ensemble (E06) was meaningful before E08; it's roughly tied now.** E06 was built on E00–E05 only (not E07/E08). It beats every original single engine and still has the lowest FP.mean (1.93) and highest precision (0.973) on this dataset. But E08 alone matches E06 on F1.median (0.973) and is within 0.2 pp on F1.mean — at ~1/5 the inference cost (E06 requires running all five upstream engines). The case for the ensemble narrows: E08 alone is cheaper and almost as accurate, except where wrong-value substitution is specifically costly and the extra precision matters. A future E06-with-E07-E08 ensemble would likely move the headroom story but is not built in this report.

9. **Some of the apparent weak spots are measurement artifacts, not engine deficiencies.** Signature (0.65 across every engine) and ~~explain_changes~~ (0.65) are being scored as strict exact-text matching, which is the wrong rule for these two categories. Signature should be presence/absence; freeform text should be fuzzy. Re-scoring these two categories under appropriate rules would lift every engine's aggregate F1 noticeably — the engines are doing better than the numbers suggest on these fields.
10. **Production behaviour will differ from these numbers.** The 40-sample dataset is small and skews toward known-hard edge cases (blank/coffee/pencil samples included by design to test robustness). On a larger and more representative production corpus, expect the relative rankings to hold but the absolute numbers to move — both up (more typical samples will be easier) and down (more unfamiliar edge cases will appear). Use these numbers as a directional guide to engine selection, not as a guarantee of production accuracy.

Appendix A — E06 ensemble combiner

This appendix documents the per-field weighted-majority combiner that produces the E06 row in the comparison tables above. The full source data and benchmark export live in [../06-engine-ensemble/](#).

Production note (read this first)

E06 is NOT a deployment. It uses predictions that were already produced by E00–E05; running E06 against new documents means running all five upstream engines first and then combining their outputs. That is ~5× the inference cost of any single engine. The ensemble is documented as a **measurement artifact** showing the headroom that exists above the best single engine — the recommended deployable path is still one of E03 or E05 alone.

Strategies explored

Six deployable strategies plus one oracle baseline ([../06-engine-ensemble/scripts/build-ensemble.py](#)):

code	how it picks
S1_per_category_best	per field's category, take the per-category best engine (E03 for phone/checkboxes, E05 for the other six). No fallback.
S2_best_then_majority_fallback	S1, but if the best engine returns null-like, fall back to a ≥3 majority vote.
S3_majority_then_best	If ≥3 engines agree on a non-null value, use it. Else, fall back to per-category best.
S4_weighted_majority	Weighted vote: each engine's vote weight = its per-category accuracy on this field's category. Pick the highest-weighted value.
S5_weighted_with_null_preference	S4, but if the per-category best engine returned null AND any other engine agrees null, prefer null (avoids over-extraction).
S6_per_field_weighted_majority	Same as S4 but with per-field weights instead of per-category — finer granularity. Chosen.
Z_oracle_upper_bound	Cheating baseline: if any engine got the field right, take that engine's value. Headroom measurement only.

Results — strategies vs single-engine baselines

Caveat — historical numbers. The single-engine rows in this Appendix table use the predictions snapshot that the ensemble was actually *built on* (improve/03 era; E03 F1.mean 0.947). The headline aggregate table at the top of this report uses the *current* canonical numbers (E03 F1.mean 0.938). The ensemble itself has not been rebuilt — that would change S1 – S6 figures here. Treat the comparisons in this Appendix as a snapshot of the ensemble's relative value at build time.

strategy	F1.median	F1.mean	Precision.mean	Recall.mean	matched.median	FP.mean
E00 alone	0.939	0.903	0.917	0.899	66	5.60
E02 alone	0.959	0.918	0.941	0.902	69	4.05
E03 alone (snapshot)	0.969	0.947	0.958	0.939	70	3.00
E04 alone	0.903	0.870	0.876	0.866	66	8.48
E05 alone	0.960	0.942	0.951	0.935	71	3.38
S1_per_category_best	0.969	0.953	0.962	0.947	71	2.65
S2_best_then_majority_fallback	0.969	0.954	0.962	0.947	71	2.65
S3_majority_then_best	0.973	0.959	0.970	0.950	71	2.15
S4_weighted_majority	0.973	0.962	0.973	0.953	71	1.93
S5_weighted_with_null_preference	0.973	0.962	0.973	0.952	71	1.90
S6_per_field_weighted_majority	0.973	0.962	0.973	0.953	71	1.93
Z_oracle_upper_bound (cheating)	0.986	0.984	0.986	0.983	73	1.03

S6 edges out S4 and S5 by a hair on most aggregates. S3/S4/S5/S6 are all within 0.2 pp of each other — any of them would be a reasonable production choice. The interesting result is that **every ensemble strategy beats every single engine on precision** (S1 = 0.962 vs E03 = 0.958), which is where the F1 lift comes from: the ensemble doesn't extract many more correct fields than E03 (matched.median 71 vs 70, just +1 field), but it makes meaningfully fewer wrong-value substitutions (FP.mean ~2 vs E03's 3).

Full strategy CSV: [../06-engine-ensemble/data/strategy-comparison.csv](#).

Why per-category-best alone (S1/S2) underperforms

S1 and S2 are the most defensible-looking strategies — pick the engine you know is best at this category. They do beat the best single engine on F1.mean (0.953 vs E03 0.947), but they trail the weighted-majority strategies (S3+) by 0.6–1 pp. The problem: the "per-category best" engine is best *on average* over that category, but on individual fields where another engine reads the form better, S1 takes the wrong answer. Voting across engines (S3 onwards) recovers those wins without sacrificing the category-leader's average advantage.

The lesson: when you have multiple roughly-equal engines, **agreement is a stronger signal than category-level ranking**. The weighted-majority strategies treat per-category accuracy as a Bayesian prior, then update it with the actual votes; that beats trusting the prior unconditionally.

Where the ensemble beats every single engine

category	E06	best single	engine	delta
sin	0.898	0.923	E05	-2.5 pp
date	0.936	0.936	E05	tie
phone	0.963	0.963	E03	tie
name	0.880	0.880	E05	tie
signature	0.680	0.675	E05	+0.5 pp
freeform_text	0.700	0.650	E05	+5.0 pp
checkboxes	0.989	0.975	E03	+1.4 pp
income_amounts	0.953	0.951	E05	+0.2 pp

The per-category accuracy is computed on the matched-field count only (it doesn't penalise substitution-FPs the way the per-sample precision does). The ensemble wins or ties on 7 of 8 categories. The single category where E06 underperforms meaningfully is **sin**, where E05 alone hits 0.923 but E06 lands at 0.898. The mechanism: on a handful of samples, E05 reads the SIN correctly but every other engine misreads it, so the weighted majority votes against E05.

The bigger story is in the **aggregate precision and FP.mean**: the ensemble cuts FP.mean from E03's 3.00 (and E04's 8.48) to 1.93 — roughly halving the substitution-error count. That's where the F1 gains come from.

Headroom — the oracle baseline

The oracle baseline cheats: for every field, it asks "did any engine got this right?" and takes that engine's answer. It is not deployable but tells us the upper bound any router could achieve on these five engines' predictions.

	best single (E03)	E06 (S6)	Oracle
F1.median	0.969	0.973	0.986
F1.mean	0.947	0.962	0.984
precision.mean	0.958	0.973	0.986
recall.mean	0.939	0.953	0.983
matched.median	70	71	73
FP.mean	3.00	1.93	1.03

E06 closes about **40% of the F1.mean gap** between the best single engine (0.947) and the oracle (0.984). The remaining 60% is on the table for a smarter router — primarily through better per-field confidence scoring, possibly with cross-engine confidence calibration. The matched-fields gap (E03 = 70, Oracle = 73 of 74) means there exist 3 fields per sample at the median where *one of the five engines* got it right but the others didn't, and our ensemble didn't pick the right one. The FP gap (1.93 → 1.03) shows the same headroom: the ensemble already roughly halves the wrong-answer rate; an oracle would halve it again.

E06 residual errors

The per-sample mismatch table for E06 is at [../06-engine-ensemble/iteration/errors-for-gt-cleanup.md](#) — **140 mismatches across 33 samples** (7 samples have zero mismatches under strict eval). This is the lowest mismatch count of any engine on this dataset:

engine	mismatches	samples with ≥ 1 mismatch
E03 (CU + gpt-5.2)	184	39
E05 (VLM + DI hybrid)	188	34
E02 (Mistral / Foundry)	297	38
E00 (DI custom template)	364	39
E04 (gpt-5.4 VLM)	377	39
E06 (ensemble)	140	33

The ensemble's structural strength shows up not just in fewer total mismatches but also in fewer samples that have any mismatch at all — 7 samples come back perfectly matched.

The residual error categories on E06 break down into:

- **Single-character handwriting** (X-marks, isolated 0 s, signature placeholders). When 4+ engines misread the same character, voting can't recover.
- **Numeric blank-vs-zero ambiguity** on a small handful of income-amount fields where the form has a stray pen mark visible. Multiple engines extract 0 ; the GT is "" .
- **One-of-array GT not yet covering an engine's format variant.** Caught by `promote-gt-format-variants.ts` and absorbed in subsequent GT cleanup passes.
- **Genuine OCR misreads** — 5 vs 8 , 1 vs 7 confusions on dense handwriting. The irreducible per-engine errors that no ensemble can fix.